

Developer Brief: Make the Publishing System Ready for Use

Personal Brand Engine — Eng. Yahya Alsalamah | Date: 2026-07-21 | Repository: `invetechs1/Self-branding` | Branch: `claude/social-media-marketing-system-oq5isu`

Goal of this brief: everything the developer must do so the owner can use the system with zero technical steps: review generated drafts, mark them `approved` , and have them auto-published to X (Twitter) and LinkedIn on schedule.

1. System Architecture (what you are integrating)

This is **not** a frontend/backend web app. It is a lightweight, serverless content-operations system that runs entirely inside GitHub:

Component	File	Role
Content generator	<code>automation/generate_posts.py</code>	Generates a full week of content (X threads, daily tweets, LinkedIn posts, articles, video scripts) via the Anthropic API, sourced strictly from <code>profile/facts.yml</code> . Output goes to the queue as drafts.
Publishing queue	<code>automation/queue/queue.csv</code>	One row per post: date, time, platform, content, status (<code>draft</code> / <code>approved</code> / <code>posted</code>). The owner's only job is flipping <code>draft</code> → <code>approved</code> .
Publisher	<code>automation/publish.py</code>	Publishes due <code>approved</code> rows — X via <code>tweetpy</code> (thread support included), LinkedIn via REST API. Never publishes drafts.
Scheduler	<code>.github/workflows/publish.yml</code>	GitHub Action, hourly cron. Builds the runtime config from GitHub Secrets and runs the publisher.
Website	<code>site/index.html</code>	Static personal site (SEO, Schema.org Person). To be served via GitHub Pages.

Flow: generate → human review & approve in the queue → auto-publish on schedule → queue status updated and committed back.

2. Your Deliverables (definition of "ready")

Deliverable	
1	X (Twitter) developer app created on the owner's account; 4 credentials generated with Read and Write permission.
2	LinkedIn developer app created; Access Token (<code>w_member_social</code>) + Person URN obtained.
3	All 7 secrets added to GitHub Actions Secrets with the exact names in Section 5.
4	Local end-to-end test passed: one real tweet + one real LinkedIn post published via the scripts (Section 6).
5	Branch merged into <code>main</code> , hourly workflow verified via a manual <code>workflow_dispatch</code> run.
6	GitHub Pages enabled for the site; domain connected; site added to Google Search Console.
7	A short handover note to the owner: how to approve posts and confirmation that everything runs (never include secret values in it).

3. X / Twitter Integration — Step by Step

1. Create a developer account at `developer.x.com` , signed in as **the owner's X account** (posts are published by whichever account owns the access tokens).
2. Plan: start with the **Free tier** (up to 500 posts/month at app level — enough for our ~60 posts/month rhythm). Upgrade to Basic only if read access or higher limits become necessary.
3. In the Developer Portal create a Project, then an App.
4. **Before generating tokens:** App settings → *User authentication settings* → enable OAuth 1.0a and set App permissions to **Read and Write**. (Tokens generated before this change are read-only and posting fails with HTTP 403.)
5. From *Keys and Tokens*, generate and store:
 - API Key + API Key Secret (a.k.a. Consumer Keys)
 - Access Token + Access Token Secret (for the owner's account, Read and Write)

The publisher uses OAuth 1.0a user context with `POST /2/tweets` ; threads are posted as reply chains automatically.

4. LinkedIn Integration — Step by Step

1. Create an app at `linkedin.com/developers` . It must be linked to a LinkedIn Company Page — create one for e.g. "Bassir Technology" if none exists.
2. Under *Products*, request both:
 - **Share on LinkedIn** → grants scope `w_member_social` (post as the member)

- **Sign In with LinkedIn using OpenID Connect** → grants `openid profile` (needed to fetch the member id)
3. Run the OAuth 2.0 (3-legged) flow as the owner with scopes `openid profile w_member_social` and obtain the **Access Token**. The portal's Token Generator tool is the quickest path.
 4. Fetch the **Person URN**:

```
curl -H "Authorization: Bearer {ACCESS_TOKEN}" https://api.linkedin.com/v2/userinfo  
# take the "sub" field → URN is: urn:li:person:{sub}
```

Token lifetime: LinkedIn access tokens expire after ~60 days. Either enable the Refresh Token program (if available to the app) or document a monthly manual renewal procedure and update the GitHub Secret accordingly. This maintenance duty is part of the handover note.

5. GitHub Secrets — Exact Names

Path: Repo → Settings → Secrets and variables → Actions → New repository secret

Secret name	Value
<code>ANTHROPIC_API_KEY</code>	Anthropic API key for content generation — from <code>console.anthropic.com</code>
<code>X_API_KEY</code>	X API Key (Consumer Key)
<code>X_API_SECRET</code>	X API Key Secret
<code>X_ACCESS_TOKEN</code>	X Access Token (Read and Write)
<code>X_ACCESS_TOKEN_SECRET</code>	X Access Token Secret
<code>LINKEDIN_ACCESS_TOKEN</code>	LinkedIn OAuth2 access token
<code>LINKEDIN_PERSON_URN</code>	Format: <code>urn:li:person:xxxx</code>

Non-negotiable security rule: no credential is ever committed to the repository. `automation/config.yml` is git-ignored and local-only; the Action rebuilds it from Secrets at runtime. Any key accidentally pushed to git must be revoked immediately and regenerated. Never paste secret values into chats, tickets, or the handover note.

6. Local End-to-End Test (before go-live)

```
# 1) Requirements (Python 3.11+)
pip install -r automation/requirements.txt

# 2) Local config
cp automation/config.example.yml automation/config.yml
#   fill in the keys inside config.yml

# 3) Generate one week of draft content into automation/queue/queue.csv
python automation/generate_posts.py --week

# 4) Approve one test row: change status draft → approved,
#   set its date/time to the recent past

# 5) Dry run (publishes nothing, shows what would run):
python automation/publish.py --dry-run

# 6) Real test publish (one tweet + one LinkedIn post):
python automation/publish.py
```

Acceptance criteria: the tweet appears on the owner's X account, the post appears on LinkedIn, and both rows flip to `posted` in the queue.

7. Go-Live Steps

1. **Merge the branch into `main`** . Critical: GitHub scheduled workflows (cron) only run from the default branch — nothing publishes automatically until this merge happens.
2. Ensure Actions are enabled for the repo, then trigger the workflow once manually (Actions tab → `publish-approved-posts` → Run workflow) and verify a green run.
3. **Website**: enable GitHub Pages (Settings → Pages), serve the `site/` content, connect the custom domain, then register the site in Google Search Console and submit the sitemap.

8. Explicitly Out of Scope (by design — do not build)

- **Instagram & TikTok auto-posting**: the system generates texts/scripts only; design, filming and posting are manual for now. (Meta Graph API / TikTok Content Posting API are an optional later phase; config fields already exist.)
- **Auto-replies, auto-follow, engagement bots**: strictly forbidden — they violate platform terms and get accounts suspended.
- Do not raise the posting frequency limits configured in the system.

9. Reference Files in the Repository

File	Contents
automation/README.md	Full system flow and per-platform requirements table
automation/config.example.yml	Config template with every required field
.github/workflows/publish.yml	Scheduler definition; builds config from Secrets
profile/facts.yml	Single source of truth for content — developer must not edit
content/drafts/00-launch-pack.md	Ready launch content (bios, pinned thread, about page)
docs/dev-integration-brief.pdf	Arabic version of this brief

Prepared from repository **invetechs1/Self-branding**, branch **claude/social-media-marketing-system-oq5isu**.
When done, confirm to the owner: secrets added (names only, never values), test results of Section 6, and the green scheduled run of Section 7.